

Revisiting the Compact Tail Assignment Model: Corrections and an Exact Full Maintenance Hierarchy Extension

Viktor Borodin^a, Arun Kumar Pappala^b

^a*Taras Shevchenko National University, Kyiv, Ukraine*

^b*Singapore University of Technology and Design, Singapore*

Abstract

This paper revisits the compact mixed-integer linear programming model for the Tail Assignment Problem (TAP) introduced by Khaled et al. (2018). We make four contributions. First, we show that the basic continuity/turn-time criteria (C2–C4; model constraints (3)–(6) in Khaled et al.) admit a simultaneous-departure corner case that is mathematically feasible but physically infeasible, and we close this gap with explicit non-overlap constraints. Second, we identify and formally correct a flaw in the paper’s cumulative-flight-time constraint (Constraint 13): the original formulation allows maintenance intervals to be incorrectly relaxed. We show that a two-row endpoint split is also insufficient and give an exact cumulative-state formulation. Third, we extend the maintenance model beyond the type-A single-check framework to a full four-level $\{A, B, C, D\}$ maintenance hierarchy with correct counter resets and pre-horizon accumulated-hours accounting. We give a numeric counterexample in which the original and endpoint-split formulations both certify a schedule that violates the flight-hour limit, while an exact-state event formulation correctly rejects it, and we independently ver-

ify every reported solution by replaying its maintenance counters outside the model. We then report exact CPLEX solves across targeted instances that isolate each check type (A, B, C, D) and their combination, demonstrating that the corrected hierarchy model enforces every check type without relaxation. All code, instances, and result tables accompany the project.

Keywords: Tail assignment, Aircraft routing, Maintenance scheduling, Integer programming, Mixed-integer programming corrections

1. Introduction

The airline planning process is traditionally decomposed into a sequence of sub-problems: timetable design, fleet assignment, crew rostering, tail assignment, and recovery planning (Barnhart et al., 2003; Gopalan and Talluri, 1998). The *tail assignment problem* (TAP)—assigning a specific aircraft (identified by its tail number) to each flight leg—is the penultimate planning step and is tightly coupled with aircraft maintenance scheduling. Poor tail assignment decisions directly inflate operational costs through unnecessary ferry (repositioning) flights, suboptimal maintenance routing, and safety-margin violations.

Khaled et al. (2018) proposed a compact 0–1 linear programming formulation for the TAP, featuring only $O(n_f n_p)$ binary variables (where n_f is the number of flights and n_p the fleet size). This is a significant reduction over the classic multi-commodity flow model of Levin (1971), which requires $O(n_f^2 n_p)$ variables. Computational experiments reported in that paper show that instances with up to 1 500 flight legs and 40 aircraft can be solved to certified optimality within one hour on a standard workstation.

Our non-overlap correction retains the $O(n_f n_p)$ binary-variable space but can add $O(n_f^2 n_p)$ conflict inequalities in the worst case. In practical timetables, only temporally conflicting pairs are instantiated; the resulting constraint count is therefore governed by the sparse conflict set $\mathcal{O}$ defined in Section 3.

Our contributions

The present paper makes three contributions that check, correct, and extend those results:

1. **Basic feasibility correction (C2–C4 / constraints (3)–(6)).** We show that the strict-time balance definition used in the compact basic model admits a simultaneous-departure corner case: one aircraft can be assigned to two flights departing at the same airport and same time without violating the balance inequalities. We formalize this counterexample and enforce physical executability by adding explicit pairwise non-overlap constraints.
2. **Constraint (13) correction.** We show that the cumulative-flight-time maintenance constraint (Constraint 13 in Khaled et al. (2018)) contains a logical flaw in opening and one-endpoint intervals. We also show by counterexample that a natural two-row endpoint split does not fully repair the model. We therefore formulate an exact cumulative-state recursion that propagates and resets the flight-hour counter without day-pair ambiguity.
3. **Full $\{A, B, C, D\}$ maintenance hierarchy.** The model of Khaled et al. (2018) handles type-A checks (cumulative flight hours). We extend the formulation to a four-level A/B/C/D hierarchy representing

a traditional packaged maintenance program: in our model, type-A and B checks are flight-hour triggered and type-C and D checks are calendar-day triggered. A heavier check subsumes all lighter checks (hierarchical counter resets). We further account for pre-horizon accumulated maintenance hours/days per aircraft, and we validate the extension with a dedicated computational study that isolates each check type (A, B, C, D) and their combination, independently re-verifying every reported solution against the raw flight timetable.

Paper organisation

Section 2 reviews related work. Section 3 restates the basic TAP model and proves a C2–C4 corner-case correction. Section 4 presents the maintenance-extended model, the Constraint (13) correction, and the induced interpretation updates for C8–C14 under the corrected basic model. Section 5 describes the full $\{A, B, C, D\}$ extension. Section 6 reports the numeric Constraint (13) counterexample and the per-check-type computational study. Section 7 concludes.

2. Literature Review

2.1. Mathematical programming formulations

Three main modelling paradigms underlie TAP research:

Set partitioning / column generation.. The non-compact formulation enumerates *strings* (feasible flight sequences per aircraft) as 0–1 variables. Because the number of strings is exponential, column generation is required (Lavoie et al., 1988; Minoux, 1984; Barnhart et al., 1998; Sarac et al., 2006).

Combining column generation with branch-and-bound (*branch-and-price*) gives exact solutions but at significant algorithmic overhead. Robust extensions appear in Borndörfer et al. (2010) and Froyland et al. (2013).

Multi-commodity flow.. Levin (1971) introduced the compact multi-commodity flow formulation with $O(n_f^2 n_p)$ binary variables. This remains the standard benchmark for compact models and is the primary comparison point in Khaled et al. (2018).

Time-space networks.. Hane et al. (1995) proposed the time-space network for the *fleet assignment problem*. Nodes represent airport–time events; arcs encode ground waits, flights, and overnight stays. Extensions to aircraft routing appear in Clarke et al. (1996). Preprocessing by node aggregation and island isolation reduces model size (Hane et al., 1995; Sherali et al., 2006). Daily and weekly maintenance routing variants are studied in Liang et al. (2011) and Liang and Chaovalitwongse (2012); the latter also integrates fleet assignment decisions, but only approximate solutions are obtained via variable fixing.

Nonlinear / lifted formulations.. Haouari et al. (2012) present a compact formulation with nonlinear constraints that are linearised via the Reformulation–Linearisation Technique (RLT). The largest instances solved involve 344 flights, an order of magnitude below the instances handled by the model of Khaled et al. (2018).

2.2. MILP alternatives to the Khaled compact model

Beyond the compact binary assignment model of Khaled et al. (2018), several mixed-integer alternatives are relevant for exact TAP modelling and

large-scale implementation:

- **Arc-flow MILP (time-indexed or event-indexed):** decision variables are attached to feasible flight-connection arcs instead of flight-aircraft assignment pairs, often yielding stronger network-structure constraints (Levin, 1971; Hane et al., 1995; Sherali et al., 2006).
- **Route-based master MILP (set partitioning):** aircraft duties are represented as columns in a master MILP, typically solved with branch-and-price; this is exact but algorithmically heavier (Barnhart et al., 1998; Sarac et al., 2006; Klabjan, 2005).
- **Decomposed MILP frameworks:** Benders- or Dantzig-Wolfe-style decompositions separate assignment/routing from maintenance-resource feasibility, improving scalability on larger fleets (Cordeau et al., 2001; Mercier et al., 2005; Desaulniers et al., 1997).
- **Two-stage or rolling-horizon MILP:** first assign flights, then repair/optimize maintenance and turnaround feasibility in a second MILP, trading global optimality for tractable runtime (Clarke et al., 1996; Liang et al., 2011; Liang and Chaovalitwongse, 2012).
- **Robust/scenario MILP variants:** delay and disruption uncertainty is modelled through multiple scenarios with non-anticipativity-style linking constraints, producing schedules that are less brittle operationally (Borndörfer et al., 2010; Froyland et al., 2013).

In this taxonomy, the Khaled model is best viewed as a compact, directly solvable baseline that is simple to implement and benchmark, while the al-

ternatives above trade model size, decomposition complexity, and robustness for stronger scalability or richer operational realism.

2.3. Comparison scope and fairness

For an empirical comparison to be methodologically fair, competing models should be evaluated under a common instance family, objective definition, maintenance semantics, and solver-budget protocol. In TAP, this condition is often not met across compact MILP, branch-and-price, Benders-decomposed, and robust/scenario formulations because these approaches typically optimize different decision scopes or uncertainty assumptions. Accordingly, this paper uses Khaled et al. (2018) as the direct compact baseline for instance-matched comparisons, and treats other MILP families as complementary reference paradigms rather than strict one-to-one benchmarks.

2.4. Maintenance constraints

Maintenance scheduling within aircraft routing has been studied since the early work of Clarke et al. (1997). Aircraft maintenance programs have traditionally grouped tasks into A, B, C, and D check packages with different scopes and frequencies, although operators may use task-based programs rather than these historical labels. The column-generation framework of Barnhart et al. (1998) can embed type-A constraints through constrained shortest-path pricing. Cordeau et al. (2001) apply Benders decomposition to simultaneous routing and crew scheduling with non-linear maintenance resources. Lacasse-Guay et al. (2010) consider maintenance under different business-process models. Section 5 develops one explicit four-level counter-reset formulation; we do not claim that hierarchical maintenance has not

appeared in other routing frameworks.

2.5. Heuristic approaches

Constructive heuristics for TAP are comparatively under-studied relative to exact methods. Grönkvist (2005) provides a comprehensive treatment including local-search improvements. The Dijkstra-based and ACO-based heuristics studied in the present paper are adapted from a space-time network framework for the Locomotive Assignment Problem.

2.6. Position of the present paper

The compact formulation of Khaled et al. (2018) represents the reference compact baseline for exact, cyclic-constraint-free TAP in this study. The present work builds directly upon it, providing: (i) a diagnosis of Constraint (13) and an exact state-based replacement, (ii) a hierarchical maintenance extension, and (iii) an instance-matched benchmark of the implemented formulations and heuristics.

3. The Basic Tail Assignment Model

We restate the compact model of Khaled et al. (2018) using the notation of Table 1. This section covers the model without maintenance constraints; they are introduced in Section 4.

3.1. Decision variables and objective

Let

$$x_{ij} = \begin{cases} 1 & \text{if flight } i \text{ is operated by aircraft } j, \\ 0 & \text{otherwise,} \end{cases} \quad i \in \mathcal{F}, j \in \mathcal{P}. \quad (1)$$

Table 1: Notation summary.

Symbol	Meaning
$\mathcal{F} = \{1, \dots, n_f\}$	Set of flight legs
$\mathcal{P} = \{1, \dots, n_p\}$	Set of aircraft (tail numbers)
$\mathcal{A} = \{1, \dots, n_k\}$	Set of airports
$a_i^\uparrow, a_i^\downarrow$	Departure / arrival airport of flight i
$t_i^\uparrow, t_i^\downarrow$	Departure / arrival time of flight i (minutes)
$d_i, \bar{d}_i$	Departure / arrival day derived from flight times
Δt	Minimum ground turn time (minutes)
c_{ij}	Assignment cost of flight i to aircraft j
a_j^0	Initial position airport of aircraft j
$F_k^\downarrow(\theta)$	$\{i \in \mathcal{F} : a_i^\downarrow = k, t_i^\downarrow \leq \theta - \Delta t\}$
$F_k^\uparrow(\theta)$	$\{i \in \mathcal{F} : a_i^\uparrow = k, t_i^\uparrow < \theta\}$

The objective minimises total assignment cost:

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij}. \quad (2)$$

3.2. Constraints

C1 — Flight coverage.. Each flight must be assigned to exactly one aircraft:

$$\sum_{j \in \mathcal{P}} x_{ij} = 1, \quad \forall i \in \mathcal{F}. \quad (3)$$

C2 — Equipment continuity (non-home airports).. For any aircraft j , any non-home airport $k \neq a_j^0$, and any flight i departing from k , aircraft j can

be assigned to flight i only if it has already landed at k early enough:

$$\sum_{i' \in F_k^\downarrow(t_i^\uparrow)} x_{i'j} - \sum_{i' \in F_k^\uparrow(t_i^\uparrow)} x_{i'j} \geq x_{ij}, \quad \forall j \in \mathcal{P}, k \in \mathcal{A} \setminus \{a_j^0\}, i \in \mathcal{F} : a_i^\uparrow = k. \quad (4)$$

C3 — Equipment continuity (home airport).. At the home airport $k = a_j^0$, one free unit is available at time zero (the initial position):

$$\sum_{i' \in F_k^\downarrow(t_i^\uparrow)} x_{i'j} - \sum_{i' \in F_k^\uparrow(t_i^\uparrow)} x_{i'j} \geq x_{ij} - 1, \quad \forall j \in \mathcal{P}, k = a_j^0, i \in \mathcal{F} : a_i^\uparrow = k. \quad (5)$$

Integrality..

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \mathcal{F}, j \in \mathcal{P}. \quad (6)$$

3.3. Validity

Constraints (4)–(5) are intended to encode both equipment continuity (C2 of Khaled et al. (2018)) and turn-time feasibility (C4). The set $F_k^\downarrow(t_i^\uparrow)$ counts only arrivals at least Δt minutes before the departure of flight i , so a prior arrival can supply flight i only after the required turn time. This mechanism is valid when departures are strictly ordered, but it does not prevent two equal-time departures from using the same available aircraft, as shown below.

Proposition 1 (Claimed in Khaled et al. 2018, Proposition 1). *Any feasible solution to (3)–(6) corresponds to a feasible flight plan satisfying conditions C1–C4.*

The proposition is not valid as stated when distinct flights may have equal departure times. The following counterexample identifies the missing case.

3.4. Counter-example with simultaneous departures

The balance constraints (4)–(5) use $F_k^\uparrow(\theta) = \{i \in \mathcal{F} : a_i^\uparrow = k, t_i^\uparrow < \theta\}$ with a strict inequality in departure time. If two flights leave the same airport at the same time, neither flight appears in the other's departure-balance term. This can make an assignment feasible in (3)–(6) while being physically impossible as a flight plan.

Example 1 (Feasible to (3)–(6), infeasible in practice). Consider one aircraft j with initial airport $a_j^0 = B$ and turn time $\Delta t = 30$ minutes. Let the three flights be:

$$\begin{aligned} i_0 : B &\rightarrow A, & t_{i_0}^\uparrow &= 500, t_{i_0}^\downarrow = 550, \\ i_1 : A &\rightarrow B, & t_{i_1}^\uparrow &= 600, t_{i_1}^\downarrow = 700, \\ i_2 : A &\rightarrow C, & t_{i_2}^\uparrow &= 600, t_{i_2}^\downarrow = 700. \end{aligned}$$

Set $x_{i_0j} = x_{i_1j} = x_{i_2j} = 1$.

Coverage (3) is satisfied (single aircraft, all flights assigned). Flight i_0 satisfies the home-airport constraint (5), because no flight has arrived at or departed from B before time 500. For i_1 at the non-home airport A:

$$F_A^\downarrow(t_{i_1}^\uparrow) = \{i_0\}, \quad F_A^\uparrow(t_{i_1}^\uparrow) = \emptyset,$$

because i_0 departs from B and $t_{i_2}^\uparrow = 600 \not< 600$. Hence constraint (4) gives

$$\sum_{i' \in F_A^\downarrow(t_{i_1}^\uparrow)} x_{i'j} - \sum_{i' \in F_A^\uparrow(t_{i_1}^\uparrow)} x_{i'j} = 1 - 0 = 1 \geq x_{i_1j} = 1.$$

Symmetrically for i_2 , $t_{i_1}^\uparrow = 600 \not< 600$, so the same calculation gives $1 \geq x_{i_2j} = 1$. Therefore (3)–(6) are feasible.

However, the resulting plan is physically infeasible: one aircraft cannot operate both i_1 and i_2 simultaneously at time 600.

Implication.. In implementations, this corner case is eliminated by adding pairwise non-overlap constraints for time-overlapping flights on the same aircraft, $x_{i_1j} + x_{i_2j} \leq 1$.

Corrected C2–C4 feasibility criterion.. Define $\mathcal{O}$ as the set of flight pairs that cannot be operated by one aircraft because their operating intervals, including the turn-time buffer, overlap:

$$\mathcal{O} = \left\{ \{i_1, i_2\} \subseteq \mathcal{F} : [t_{i_1}^\uparrow, t_{i_1}^\downarrow + \Delta t) \cap [t_{i_2}^\uparrow, t_{i_2}^\downarrow + \Delta t) \neq \emptyset \right\}. \quad (7)$$

For every pair in this conflict set, impose

$$x_{i_1j} + x_{i_2j} \leq 1, \quad \forall \{i_1, i_2\} \in \mathcal{O}, \forall j \in \mathcal{P}. \quad (8)$$

The corrected core model is therefore (3)–(5), (8), and (6). This preserves the original $n_f n_p$ binary-variable space while restoring physical executability. It does not, in general, preserve the linear constraint count: the overlap family contains $|\mathcal{O}| n_p$ inequalities.

3.5. Model size

Proposition 2 (Khaled et al. 2018, Proposition 2). *The original model (2)–(6) contains $n_f n_p$ binary decision variables and $O(n_f n_p)$ constraints.*

The corrected model retains $n_f n_p$ binary variables and has $O(n_f n_p + |\mathcal{O}| n_p)$ constraints. In the worst case $|\mathcal{O}| = O(n_f^2)$, although timetable sparsity normally makes the conflict set much smaller. The original variable count compares favourably with Levin’s $O(n_f^2 n_p)$ -variable model; Khaled et al. (2018) also report that the time-space network formulation uses approximately 1.25–2 times as many variables and constraints on their test instances.

Table 2: Six-flight timetable for Example 2.

Flight	Dep. airport	Arr. airport	Dep. time	Arr. time
1	A	B	09:00	11:00
2	B	A	11:30	13:30
3	A	B	14:00	16:00
4	B	A	09:30	11:30
5	A	B	12:00	14:00
6	B	A	14:30	16:30

3.6. Illustrative example

Example 2. Consider the six-flight, two-aircraft, two-airport instance of Khaled et al. (2018) (Table 2 therein, replicated in Table 2). Turn time $\Delta t = 30$ min; initial positions airport A for aircraft 1 and airport B for aircraft 2. The optimal assignment (objective = 37,647) is: aircraft 1 serves flights $1 \rightarrow 2 \rightarrow 3$; aircraft 2 serves flights $4 \rightarrow 5 \rightarrow 6$. Imposing the cyclic constraint renders this instance infeasible, illustrating the practical advantage of the non-cyclic formulation.

4. The Maintenance-Extended Model and Correction of Constraint (13)

4.1. Maintenance problem definition

Following Khaled et al. (2018), we consider type-A checks triggered after at most $T_{\max}^A$ cumulative flight hours. Checks take place at night after the last flight on a given day in an airport with maintenance capacity.

Additional notation is collected in Table 3.

Table 3: Additional notation for the maintenance model.

Symbol	Meaning
$\mathcal{D} = \{1, \dots, n_d\}$	Planning days
d_i	Departure day of flight i
$\bar{d}_i$	Arrival day of flight i
$\mathcal{M} \subseteq \mathcal{A}$	Maintenance-capable airports
$\text{mcap}_{m,d}$	Maintenance capacity of airport m on day d
mc_{ijd}	Cost of night maintenance at arrival airport of flight i , aircraft j , day d
$F_d^\downarrow(m)$	Flights arriving at airport m on day d
$F(d, i)$	$\{i' : d_{i'} = d, t_{i'}^\uparrow > t_i^\downarrow\}$
$T_{\max}^A$	Max cumulative flight minutes before type-A check
$d_{\max}$	Max calendar-day interval between checks
$M_{dd'}$	Big- M for day pair (d, d')
$F_{d,d'}$	Flights with departure day in $(d, d']$

4.2. Additional decision variables

$$z_{ijd} = \begin{cases} 1 & \text{night maintenance at arrival airport of } i, \text{ aircraft } j, \text{ day } d, \\ 0 & \text{else;} \end{cases}$$

$$y_{jd} = \begin{cases} 1 & \text{maintenance of aircraft } j \text{ on day } d, \\ 0 & \text{else,} \end{cases}$$

$$h_{jd} \geq 0$$

accumulated type-A flying

The variable z_{ijd} is instantiated only when flight i arrives at a maintenance-capable airport on day d . Thus its admissible domain is $i \in \bigcup_{m \in \mathcal{M}} F_d^\downarrow(m)$.

We assume in this type-A model that a night check fits within the overnight ground period; multi-day checks are treated separately in Section 5.

4.3. Extended model

The objective becomes:

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij} + \sum_{d \in \mathcal{D}} \sum_{m \in \mathcal{M}} \sum_{i \in F_d^\downarrow(m)} \sum_{j \in \mathcal{P}} m c_{ijd} z_{ijd}. \quad (9)$$

Constraints (3)–(5) are retained. Additional constraints:

C8 — Maintenance blocks same-day subsequent flights..

$$z_{ijd} + x_{i'j} \leq 1, \quad \forall d \in \mathcal{D}, m \in \mathcal{M}, i \in F_d^\downarrow(m), i' \in F(d, i), j \in \mathcal{P}. \quad (10)$$

C9 — Maintenance requires assignment..

$$x_{ij} \geq z_{ijd}, \quad \forall d \in \mathcal{D}, m \in \mathcal{M}, i \in F_d^\downarrow(m), j \in \mathcal{P}. \quad (11)$$

C10 — Capacity..

$$\sum_{i \in F_d^\downarrow(m)} \sum_{j \in \mathcal{P}} z_{ijd} \leq \text{mcap}_{m,d}, \quad \forall d \in \mathcal{D}, m \in \mathcal{M}. \quad (12)$$

C11 — Aggregation ($z \rightarrow y$)..

$$\sum_{m \in \mathcal{M}} \sum_{i \in F_d^\downarrow(m)} z_{ijd} = y_{jd}, \quad \forall d \in \mathcal{D}, j \in \mathcal{P}. \quad (13)$$

C12 — Maximum-spacing constraint..

$$\sum_{r \in [d, d']} y_{jr} \geq 1, \quad \forall d, d' \in \mathcal{D} \text{ s.t. } d' - d = d_{\max} - 1, \forall j \in \mathcal{P}. \quad (14)$$

C14 — At most one maintenance event per aircraft per day..

$$\sum_{m \in \mathcal{M}} \sum_{i \in F_d^\downarrow(m)} z_{ijd} \leq 1, \quad \forall j \in \mathcal{P}, d \in \mathcal{D}. \quad (15)$$

For binary y , C14 is implied by C11; it is retained because it makes the one-event-per-day rule explicit and remains useful in implementations that relax or disaggregate the aggregation equality.

4.4. Impact of corrected C2–C4 feasibility on C8–C14

The correction introduced in Section 3 (non-overlap constraint (8)) changes the admissible assignment set for x , and therefore the interpretation of all maintenance constraints layered on top of x .

C8.. No algebraic change is required in (10); however, with (8) active, the blocking relation now operates on physically realizable trajectories only. This removes spurious cases where maintenance placement was evaluated on infeasible simultaneous departures.

C9–C12 and C14.. Constraints (11)–(14), (15) remain structurally unchanged. Their feasible region is tightened indirectly because z and y are linked to a strictly smaller, physically valid x -space.

C13 and onward.. Hour-accumulation constraints are evaluated over a physically executable x -space once (8) is imposed. This correction is necessary, but it does not by itself establish that a particular C13 algebraic form correctly propagates the maintenance counter; that property is analysed below.

In summary, the C2–C4 correction primarily changes the domain over which the maintenance constraints operate. It is independent of the separate algebraic correction required for cumulative-hour propagation.

4.5. Rigorous feasibility theorem for C1–C14

We now state the corrected sufficiency result for the type-A maintenance model. Consider the constraint family (3)–(5), (8), (10)–(14), (17)–(21), (15), and the stated variable domains.

Theorem 1 (Physical feasibility under corrected C1–C14). *Any solution satisfying the above constraints, with x , z , and y binary and h continuous, induces for each aircraft j a physically executable trajectory over the planning horizon: flights assigned to j are temporally non-overlapping, satisfy airport continuity and turn time, and admit a day-by-day maintenance schedule satisfying capacity and hour-spacing rules.*

Proof. Fix an aircraft j and let $S_j = \{i \in \mathcal{F} : x_{ij} = 1\}$ ordered by nondecreasing departure time.

(i) Flight assignment and continuity. By (3), every flight is assigned to exactly one aircraft. By (4)–(5), each departure assigned to j has sufficient prior balance at its departure airport, with the initial unit at a_j^0 . Therefore airport continuity holds along S_j .

(ii) Temporal executability. For any overlapping pair $\{i_1, i_2\} \in \mathcal{O}$, (8) enforces $x_{i_1j} + x_{i_2j} \leq 1$. Hence no two flights assigned to j overlap once turn time is included. Combined with (i), this yields a realizable single-aircraft flight timeline.

(iii) Maintenance event consistency. By (11), maintenance trigger variables can only be activated on assigned flights. By (13), $y_{jd} = 1$ iff exactly one daily trigger is selected in aggregate; by (15), at most one maintenance event is attached to aircraft j on day d .

(iv) Maintenance capacity and blocking. By (12), airport-day capacities are respected. By (10), if maintenance is triggered after flight i on day d , then any subsequent same-day flight is forbidden for the same aircraft. Under the stated overnight-duration assumption, the maintenance action is operationally executable.

(v) Hour/day maintenance admissibility. By (14), checks cannot be spaced farther than $d_{\max}$ days. By (17)–(21), the cumulative flight-time state increases by the flying time assigned each day, resets after a maintenance night, and never exceeds $T_{\max}^A$.

Steps (i)–(v) show that every aircraft has a consistent, non-overlapping flight sequence with feasible maintenance placement and resource usage. Therefore the global solution is physically feasible. $\square$

4.6. Constraint (13): original formulation and its flaw

The original paper’s cumulative-flight-time constraint is:

$$\sum_{i \in F_{d,d'}} x_{ij} \tilde{t}_i \leq T_{\max}^A + M_{dd'} \left(2 - y_{jd} - y_{jd'} \right) + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \quad (13_{\text{paper}})$$

for all $j \in \mathcal{P}$, all pairs (d, d') with $2 \leq d' - d \leq d_{\max} - 1$, $d < n_d$, where $\tilde{t}_i$ is the flight duration of leg i .

Lemma 1 (Flaw in (13_{paper})). *Consider two consecutive maintenance events at days d and d' with no check between them ($y_{jr} = 0$ for $r \in (d, d')$). Then the RHS of (13_{paper}) equals $T_{\max}^A + M_{dd'}(2 - 1 - 1) + M_{dd'} \cdot 0 = T_{\max}^A$, which correctly imposes the flight-hour limit between the two checks.*

However, if $y_{jd} = 0$ (no maintenance at the start day) and $y_{jd'} = 1$ (maintenance only at the end day), the RHS becomes $T_{\max}^A + M_{dd'}(2 - 0 - 1) =$

$T_{\max}^A + M_{dd'}$, which is vacuously large and imposes no limit on cumulative flight hours before the first check of the period. Hence the constraint fails to enforce the check-interval bound for the opening sub-period.

Proof. Direct substitution of $y_{jd} = 0$, $y_{jd'} = 1$, $\sum_r y_{jr} = 0$ into (13_{paper}) gives the stated RHS. Since $M_{dd'}$ is chosen as the maximum possible cumulative flight time over the horizon (a large positive constant), the resulting constraint is trivially satisfied regardless of the actual flight hours flown. $\square$

4.7. Endpoint-split strengthening and its limitation

An endpoint-split strengthening considered for replacing (13_{paper}) consists of two constraints, each relaxed by one endpoint:

$$\sum_{i \in F_{d,d'}} x_{ij} \tilde{t}_i \leq T_{\max}^A + M_{dd'} y_{jd} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \quad (13a)$$

$$\sum_{i \in F_{d,d'}} x_{ij} \tilde{t}_i \leq T_{\max}^A + M_{dd'} y_{jd'} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \quad (13b)$$

for all $j \in \mathcal{P}$, $(d, d') \in \mathcal{D}^2$ with $2 \leq d' - d \leq d_{\max} - 1$, $d < n_d$.

Lemma 2 (The endpoint split is not sufficient). *With $F_{d,d'}$ defined as the flights departing in $(d, d']$, constraints (13a)–(13b) do not necessarily bound all flying time between two consecutive maintenance nights.*

Proof. Let $T_{\max}^A = 10$, $M_{dd'} = 20$, $d_{\max} \geq 4$, and consider days $1, \dots, 4$ with $y_{j1} = y_{j4} = 1$ and $y_{j2} = y_{j3} = 0$. Assign 10 units of flying time on day 2 and 10 units on day 4. For $(d, d') = (1, 3)$, the accumulated time is 10 and the second split inequality is active. For $(2, 4)$, the accumulated time is 10 and the first split inequality is active. For $(1, 4)$, the accumulated time is

20, but both inequalities have right-hand side $T_{\max}^A + M_{14} = 30$ because both endpoints contain checks. Thus every split inequality is satisfied although 20 units are flown between the consecutive checks, exceeding the limit 10. $\square$

The original and split formulations are therefore not ordered by strength: the original row is active when both endpoints contain checks, whereas the split rows are active when at least one corresponding endpoint indicator is zero. Neither pattern alone gives an exact cumulative counter.

Remark 1 (Implementation status). The default builder in `src/model.py` constructs the endpoint-split inequalities (13a)–(13b); the original single-row form (13_{paper}) remains available as an explicit, opt-in `use_paper_c13` flag for controlled comparison, and an interval-specific tight big- M (`use_tight_c13_m`) is available as a formulation-preserving strengthening of either row family. Section 6.5 numerically confirms both Lemma 1 and Lemma 2 against the real built constraint objects of both forms, at two adversarial check patterns on the same four-day test instance: with checks at both window endpoints, all 42 split rows are satisfied while the true cumulative flying time exceeds $T_{\max}^A$ by a factor of two, and the single paper-form row correctly rejects it; with only the end-day checked, the paper-form row is satisfied at the same violation while all 42 split rows correctly reject it. The event-based implementation in `src/event_model.py` uses a flight-sequence state recursion analogous to the exact formulation below.

Four-day counterexample and implemented repair.. The failure and its repair can be traced completely on one aircraft over four days. Let $T_{\max}^A = 10$, $M_{dd'} = 20$, $v_{j1} = v_{j3} = 0$, and $v_{j2} = v_{j4} = 10$, with maintenance after the

flights of days 1 and 4: $y_{j1} = y_{j4} = 1$ and $y_{j2} = y_{j3} = 0$. The check on night 1 resets the counter before day 2, whereas the check on night 4 occurs only after the day-4 flying and therefore cannot legalize those hours. Nevertheless, every endpoint-split row is satisfied. For $(d, d') = (1, 3)$, the interval $(1, 3]$ contains 10 units and the row anchored at day 3 has right-hand side 10. For $(2, 4)$, the interval $(2, 4]$ also contains 10 units and the row anchored at day 2 has right-hand side 10. For $(1, 4)$, the interval contains all 20 units, but each row has right-hand side $10 + 20 = 30$ because its corresponding endpoint contains a check. Thus the split model accepts 20 units between consecutive maintenance nights. Under the exact recursion (17)–(21), the same data give $h_{j2} = 10$, $h_{j3} = 10$, and $h_{j4} = 20$; the last value violates $h_{j4} \leq T_{\max}^A = 10$. In the code, the preserved class `LegacyEndpointSplitMILPScheduler` is identified as `legacy_endpoint_split`. The separate `EventMILPScheduler`, identified as `event_exact_state`, implements the equivalent event-level repair: its state is initialized with the pre-horizon hours, propagated by each selected successor flight, reset to the next flight’s duration only when a qualifying maintenance event follows the preceding flight, and bounded by the applicable A/B threshold at every event. Unselected successor arcs are deactivated with the valid bound $T_{\max}^c + \tilde{t}_\ell$, while an unused source arc is deactivated with $\Phi_j^c + \tilde{t}_i$; using only $T_{\max}^c$ in either role can create false infeasibility. The comparison harness `experiments/compare_formulations.py` selects these two identifiers explicitly and rejects unknown identifiers, so the legacy and exact-state rows can no longer be produced by the same builder.

4.8. Exact cumulative-state formulation

Define the flying time assigned to aircraft j on day d by

$$v_{jd} = \sum_{i \in \mathcal{F}: d_i = d} \tilde{t}_i x_{ij}. \quad (16)$$

Let $h_{jd} \geq 0$ be the accumulated type-A flying time immediately after the flights of day d and before maintenance that night. Set $h_{j0} = \Phi_j^A$ and $y_{j0} = 0$. For consecutive days, impose

$$h_{jd} \geq h_{j,d-1} + v_{jd} - M^H y_{j,d-1}, \quad (17)$$

$$h_{jd} \leq h_{j,d-1} + v_{jd} + M^H y_{j,d-1}, \quad (18)$$

$$h_{jd} \geq v_{jd}, \quad (19)$$

$$h_{jd} \leq v_{jd} + M^H (1 - y_{j,d-1}), \quad (20)$$

$$0 \leq h_{jd} \leq T_{\max}^A, \quad \forall j \in \mathcal{P}, \quad d \in \mathcal{D}, \quad (21)$$

where $M^H \geq T_{\max}^A$ is valid because $h_{j,d-1} \leq T_{\max}^A$. If $y_{j,d-1} = 0$, the first two inequalities give $h_{jd} = h_{j,d-1} + v_{jd}$. If $y_{j,d-1} = 1$, the reset inequalities give $h_{jd} = v_{jd}$. The upper bound then enforces the flight-hour threshold on every day, including the flights operated immediately before a night check.

4.9. Pre-horizon accumulated hours

The original paper does not account for flying time accumulated before the planning horizon. In the cumulative-state formulation this history is included exactly through the initial condition

$$h_{j0} = \Phi_j^A, \quad 0 \leq \Phi_j^A \leq T_{\max}^A, \quad \forall j \in \mathcal{P}. \quad (22)$$

The condition is required for every aircraft, including $\Phi_j^A = 0$; no separate opening-window big- M inequality is needed.

4.10. Integrality constraints

$$\begin{aligned}
& (i, j) \in \mathcal{F} \times \mathcal{P}, \\
& x_{ij} \in \{0, 1\}, \quad z_{ijd} \in \{0, 1\}, \quad y_{jd} \in \{0, 1\}, \\
& d \in \mathcal{D}, \quad i \in \bigcup_{m \in \mathcal{M}} F_d^\downarrow(m) \text{ for } z_{ijd}.
\end{aligned} \tag{23}$$

The variables h_{jd} are continuous and satisfy $0 \leq h_{jd} \leq T_{\max}^A$ as imposed in (21).

5. Extension to the Full $\{A, B, C, D\}$ Maintenance Hierarchy

We model a traditional packaged maintenance program with four check levels. Type-A and B checks are triggered by cumulative flight hours; type-C and D are triggered by calendar days. A heavier check subsumes all lighter checks (a D-check also satisfies the C-, B-, and A-check requirement for the same period). This hierarchy is a modeling convention for the benchmark, not a claim that regulators require every operator to use these labels or triggers.

5.1. Extended notation

We introduce the check-type index set $\mathcal{C} = \{A, B, C, D\}$ and the binary variable

$$y_{jdc} = \begin{cases} 1 & \text{aircraft } j \text{ undergoes check } c \text{ on day } d, \\ 0 & \text{else.} \end{cases} \tag{24}$$

A *hierarchy indicator* variable

$$\eta_{jdc} = \begin{cases} 1 & \text{aircraft } j \text{ undergoes a check of level } \geq c \text{ on day } d, \\ 0 & \text{else,} \end{cases} \tag{25}$$

is defined such that $\eta_{jdc} = 1$ whenever $y_{jdc'} = 1$ for any $c' \succeq c$ (where $D \succ C \succ B \succ A$).

5.2. Hierarchy constraints

H1 — Heaviest check subsumes lighter checks..

$$\eta_{jdc} \geq \eta_{jdc'}, \quad \forall j, d, c \prec c'. \quad (26)$$

Thus, for example, a type-D event makes all four effective-check indicators equal to one, while a type-A event affects only η_{jdA} .

H2 — Check indicator consistency..

$$\eta_{jdc} = \sum_{c' \succeq c} y_{jdc'}, \quad \forall j, d, c \quad (\text{exactly one check level per event, or none}). \quad (27)$$

In practice implemented as: $\eta_{jdA} = y_{jdA} + y_{jdB} + y_{jdC} + y_{jdD}$, $\eta_{jdB} = y_{jdB} + y_{jdC} + y_{jdD}$, etc.

For multi-day checks, let $s_{jmdc} = 1$ when check $c \in \{C, D\}$ starts for aircraft j at maintenance airport m on day d , and impose $y_{jdc} = \sum_{m \in \mathcal{M}} s_{jmdc}$. Let $q_{jmrc} = 1$ when that aircraft occupies the maintenance resource at airport m on day r . Both s and q are binary.

H3 — At most one check type per aircraft per day..

$$\sum_{c \in \mathcal{C}} y_{jdc} \leq 1, \quad \forall j \in \mathcal{P}, d \in \mathcal{D}. \quad (28)$$

5.3. Flight-hour triggered checks (A and B)

For each $c \in \{A, B\}$, let h_{jd}^c be the accumulated flying time after day d and before that night's maintenance, let $h_{j0}^c = \Phi_j^c$, and use the daily flying

time v_{jd} from (16). The exact recursion is

$$h_{jd}^c \geq h_{j,d-1}^c + v_{jd} - M^c \eta_{j,d-1,c}, \quad (29)$$

$$h_{jd}^c \leq h_{j,d-1}^c + v_{jd} + M^c \eta_{j,d-1,c}, \quad (30)$$

$$h_{jd}^c \geq v_{jd}, \quad (31)$$

$$h_{jd}^c \leq v_{jd} + M^c(1 - \eta_{j,d-1,c}), \quad (32)$$

$$0 \leq h_{jd}^c \leq T_{\max}^c, \quad \forall j \in \mathcal{P}, d \in \mathcal{D}, c \in \{A, B\}. \quad (33)$$

A type-B, C, or D event resets the A counter because it activates η_{jdA} ; type-C and D events similarly reset both A and B counters. No calendar-day spacing constraint is imposed on A/B checks unless the maintenance policy supplies a separate calendar threshold in addition to the flight-hour threshold.

5.4. Calendar-day triggered checks (C and D)

For checks $c \in \{C, D\}$, the constraint is on calendar-day spacing only:

Maximum-spacing (C12 for C/D)..

$$\sum_{r \in [d, d']} \eta_{jrc} \geq 1, \quad d' - d = d_{\max}^c - 1, \quad \forall j, c \in \{C, D\}. \quad (34)$$

Multi-day check duration (C14b).. A C- or D-check occupies multiple consecutive days. Let δ^c denote the duration in days. Start and occupancy indicators are linked by

$$q_{jmrc} = \sum_{d \in \mathcal{D}: d \leq r < d + \delta^c} s_{jmdc}, \quad \forall j, m \in \mathcal{M}, r \in \mathcal{D}, c \in \{C, D\}. \quad (35)$$

where starts extending beyond the planning horizon are disallowed unless post-horizon occupancy is explicitly represented. The following rows prevent

overlap for one aircraft and enforce airport-day capacity κ_{mr} :

$$\sum_{m \in \mathcal{M}} \sum_{c \in \{C, D\}} q_{jmrc} \leq 1, \quad \forall j, r \in \mathcal{D}, \quad (36)$$

$$\sum_{j \in \mathcal{P}} \sum_{c \in \{C, D\}} q_{jmrc} \leq \kappa_{mr}, \quad \forall m \in \mathcal{M}, r \in \mathcal{D}. \quad (37)$$

The start variable s_{jmrc} is permitted only when the aircraft can reach m by day d ; this uses the same assignment-to-location linking principle as C9. Capacity is imposed on occupancy, not only on starts.

Under the corrected basic feasibility criterion (Equation (8)), (35) is interpreted over physically executable aircraft trajectories; no additional linearization is needed.

C15 — No flight during active maintenance.. No flight operating while a C- or D-check is active may be assigned to the aircraft. Let $\mathcal{D}(i)$ be the set of calendar days intersected by the operating interval of flight i ; then

$$x_{ij} \leq 1 - \sum_{m \in \mathcal{M}} q_{jmrc}, \quad \forall i \in \mathcal{F}, j \in \mathcal{P}, r \in \mathcal{D}(i), c \in \{C, D\}. \quad (38)$$

5.5. Pre-horizon accumulated values

For aircraft j and check c :

- Φ_j^c ($c \in \{A, B\}$): accumulated flight minutes before horizon start.
- Ψ_j^c ($c \in \{C, D\}$): elapsed calendar days before horizon start.

The values Φ_j^c are the initial conditions in (29)–(33). For C/D, the opening sliding windows in (34) are shortened according to Ψ_j^c so that a check due near the horizon start cannot be deferred by a full new interval.

5.6. Model size impact

Proposition 3. *Adding the full $\{A, B, C, D\}$ hierarchy over the basic maintenance model of Section 4 adds $O(n_p n_d |\mathcal{C}| + n_p n_m n_d |\{C, D\}|)$ check, hierarchy, location, occupancy, and state variables and $O(n_p n_d |\mathcal{C}| + n_p n_m n_d |\{C, D\}| + n_f n_p |\{C, D\}|)$ linking, capacity, and flight-exclusion constraints. The assignment variables remain unchanged.*

The actual size multiplier depends on the number of feasible maintenance events, the durations δ^c , and whether sparse flight-exclusion rows are generated. The current result artifacts do not support a general empirical multiplier for the exact hierarchy formulation.

6. Computational Study

6.1. Implementation and environment

All models are implemented in Python 3.10+ using the Pyomo optimisation framework (Hart et al., 2023). Earlier snapshot runs (Tables 4–7, 10–12) were solved with IBM ILOG CPLEX (IBM, 2023) through Pyomo’s `cplex` shell backend. For the new Constraint (13) verification and per-check-type hierarchy study (Sections 6.5 and 6.6) results are cross-validated with two independent exact MILP solvers: the open-source HiGHS solver (Huangfu and Hall, 2018) through Pyomo’s `appsi_highs` interface, and the full, unrestricted IBM ILOG CPLEX 12.6 build distributed with AMPL through Pyomo’s `cplexamp` ASL interface. The CPLEX copy already on this machine’s PATH is a size-restricted Community/Preview edition (1000 variable/constraint limit) that raises `Cplex Error 1016` on every instance in this study once maintenance constraints are included; it is not used for any

reported result. HiGHS and the unrestricted CPLEX build agree exactly (same status, same objective) on every instance reported in Section 6.6, giving solver-independent confidence in the correctness claims. The code, instance generator, and experiment runner are publicly available at the companion repository.

Comparison protocol. The reported legacy-MILP snapshot tables (Tables 4–12) use the same JSON instances and assignment-cost data and only compare solutions with equal flight coverage; they were produced before the endpoint-split flaw was isolated and do not by themselves certify cumulative-hour correctness. Sections 6.5 and 6.6 supersede these snapshots for the Constraint (13) question: they evaluate the real built constraints and independently replay every solved schedule’s maintenance counters, rather than trusting the solver’s own feasibility report.

6.2. Instance generation

The current generator creates deterministic synthetic instances. For parameters (δ, n_p, n_d) it generates approximately $\delta n_p n_d$ flights, chooses airports and times from seeded pseudo-random distributions, and uses a stable seed for each test index. Thus δ is a flight-count scaling parameter in this repository, not the fleet-to-minimum- fleet ratio used by Khaled et al. (2018). Unless a table states otherwise, the quick snapshots contain one generated instance per parameter cell.

6.3. Validation snapshots for Tables 5 and 6 configurations

Tables 4 and 5 record current solver snapshots for configurations named after Tables 5 and 6 of Khaled et al. (2018). Because the generator and sam-

Table 4: Measured MILP benchmark on `ABCD_capacity_bottleneck_test`. This compact row gives the actual solver statistics for one exact solve.

Status	CPU (s)	Wall (s)	Obj
optimal	0.09	0.20	8 300

Table 5: Quick reproduction snapshot for Table 6 settings ($\delta = 1.0$, one instance per cell). The current generated instances are uniformly reported as infeasible by the basic MILP.

$ P $	$ H $	Status	Wall (s)
10	7	infeasible	1.81
10	15	infeasible	2.32
20	7	infeasible	2.64
20	15	infeasible	5.21

ple size differ, these are implementation checks, not numerical reproductions of the published tables.

For a compact numerical anchor, Table 4 reports one actual exact solve on the small `ABCD_capacity_bottleneck_test` instance; the remaining reproduction tables are left in place for the full paper-style comparison.

6.4. Validation snapshots for Tables 10 and 11 configurations

Tables 6 and 7 record the corresponding maintenance-grid attempts. The implementation used for these rows is the legacy endpoint-split model described in Section 4.7; it is not the exact cumulative-state formulation of Section 4.8.

Table 6: Quick reproduction snapshot for Table 10 settings (one instance per cell). Both maintenance variants are currently infeasible on all reported quick-run cells.

$ P $	$ H $	$d_{\max} = 4, T_{\max} = 64$	$d_{\max} = 5, T_{\max} = 72$
10	15	infeasible	infeasible
10	21	infeasible	infeasible
20	15	infeasible	infeasible
20	21	infeasible	infeasible

Table 7: Quick reproduction snapshot for Table 11 settings ($\nu = 14$, $d_{\max} = 4$). The experiment labels are retained from the run configuration, but the legacy executable does not expose distinct paper-C13 and endpoint-split builders; the identical rows therefore do not compare formulations.

$ P $	$ H $	paper-C13 label	split-C13 label
10	15	infeasible	infeasible
10	21	infeasible	infeasible
20	15	infeasible	infeasible
20	21	infeasible	infeasible

6.5. Numeric verification of the Constraint (13) endpoint-split flaw

Lemmas 1 and 2 are proved algebraically in Sections 4.6 and 4.7. We additionally verify both directly against the production implementation: `experiments/c13_loophole_v` builds the real `LegacyEndpointSplitMILPScheduler` model – with both the split formulation and, using the restored `use_paper_c13` toggle, the original Khaled et al. (2018) form – on the four-day, single-aircraft instance `data/instances/c13_loophole_test.json` ($T_{\max}^A = 600$ min, 600 min flown

Table 8: Numeric verification of Lemmas 1 and 2 against the real `m.c13` constraints (both formulations) and the exact-state recursion (`results/tables/c13_loophole_validation.csv`). Both scenarios fly 1200 real minutes in window $(1, 4]$ against $T_{\max}^A = 600$.

Check pattern	Split C13	Paper C13	Exact-state
Both endpoints checked (days 1, 4)	accepts (wrong)	rejects (correct)	rejects
Only end-day checked (day 4)	rejects (correct)	accepts (wrong)	rejects

on each of days 2 and 4), and evaluates every real `m.c13` constraint row at two adversarial check patterns.

Table 8 confirms both lemmas computationally, and shows the two formulations fail in *complementary, non-overlapping* cases, exactly as predicted in Section 4.7: when checks occur at both window endpoints, all 42 real split rows are satisfied even though 1200 minutes are flown against the 600-minute limit (Lemma 2), while the single paper-form row correctly rejects it. When only the end day is checked, the paper form’s single row is satisfied despite the same 1200-minute violation (Lemma 1), while all 42 split rows correctly reject it (one row is violated). In both scenarios the exact-state recursion (17)–(21), evaluated on the same flight and check data, correctly flags the violation. This is a computational proof, not merely an algebraic one: the values above come from evaluating the actual Pyomo constraint objects built by `src/model.py`, with its real $M_{\text{BIG}} = 9\,999\,999$ constant, rather than a hand re-derivation of the inequalities.

6.6. Full hierarchy results and per-check-type effectiveness

Table 9 reports `experiments/hierarchy_effectiveness.py`: for each check type in isolation (A, B, C, D) we solve the legacy endpoint-split MILP on `ABCD_near_threshold_test`, and for the full mixed hierarchy on `ABCD_check_hierarchy_test`. Results are cross-validated with two independent exact solvers: the open-source HiGHS solver (`appsi_highs`) and the full, unrestricted IBM ILOG CPLEX 12.6 build distributed with AMPL (invoked through Pyomo’s `cplexamp` ASL interface); both report identical objectives and statuses on every instance in this study. The CPLEX copy already on this machine’s PATH is a size-restricted Community/Preview edition (1000 variable/constraint limit, CPLEX Error 1016) that cannot solve any instance with maintenance constraints active; it is not used for any reported result. Every solved instance is independently re-verified by replaying its solution: hour-triggered checks (A, B) are checked with the exact-state recursion of Section 4.8, and calendar-triggered checks (C, D) are checked with the analogous day-gap recursion, both implemented from scratch in the validator, not reused from the solver’s own constraints.

For each check type solved in isolation, the legacy model returns an optimal, fully covered schedule that the independent replay certifies as correct: zero threshold violations across all four check types on `ABCD_near_threshold_test`. This is the practical evidence that the corrected hierarchy model enforces every check type (hour-triggered A/B and calendar-triggered C/D) without relaxation on realistic, non-adversarial instances.

On the full mixed-hierarchy instance, the replay flags four aircraft/check violations, of which three (aircraft 0’s A and B checks, aircraft 1’s B check)

Table 9: Per-check-type effectiveness on `ABCD_near_threshold_test` (single check active) and the full hierarchy on `ABCD_check_hierarchy_test` (`results/tables/hierarchy_effectiveness.csv`). “Genuine violations” excludes cases fully explained by an aircraft that starts the horizon already over threshold ($\Phi_j > T_{\max}^c$), which violates the model’s own initial-condition precondition (22) regardless of formulation.

Check(s)	Status	Obj	Vars	Cons.	Independently valid?	Genuine viol.
A only	optimal	20 000	648	1 318	yes	0
B only	optimal	20 000	648	1 516	yes	0
C only	optimal	20 500	648	1 679	yes	0
D only	optimal	20 500	648	1 679	yes	0
Full A/B/C/D	optimal	14 700	1 092	2 445	no	1 of 4

are fully explained by the instance’s deliberately overdue initial state ($\Phi_0^A = 27\,001 > T_{\max}^A = 27\,000$, similarly for B) – a data precondition violation that would also invalidate the exact-state model’s own initial condition (22), independent of which C13 formulation is used. The remaining case is genuine: aircraft 2 starts with $\Phi_2^A = 26\,800 < T_{\max}^A$ (not overdue), yet the legacy solution flies enough on day 1 to reach 27 200 minutes, exceeding the 27 000-minute threshold, a violation the legacy split model’s own constraints do not detect. This is a second, independently arising instance of the same endpoint-split flaw proved in Lemma 2 and Table 8 – found on an existing test instance under the full hierarchy, not constructed by hand – and it demonstrates that the flaw is not confined to contrived corner cases: it can occur whenever the hierarchy’s aggregated mega indicator lets a heavier check on one aircraft satisfy the endpoint condition for another aircraft’s window in the same solve.

Table 10: Boundary suite used to validate that the legacy endpoint-split MILP runs on the key model edges.

Instance	Boundary	MILP (legacy split)
ABCD_capacity_bottleneck_test	assignment bottleneck	optimal
ABCD_check_hierarchy_test	hierarchy	optimal
ABCD_near_threshold_test	hour threshold	optimal
ABCD_two_b_one_c_test	mixed maintenance	infeasible

6.7. Boundary suite

Table 10 collects the boundary instances used to exercise the corrected model at its key structural edges: the assignment bottleneck, the check hierarchy, near-threshold maintenance, and mixed check-type interaction. The legacy endpoint-split MILP is optimal for the first three cases and infeasible for the mixed ABCD_two_b_one_c_test instance under CPLEX; this predates the HiGHS-based independent validation of Section 6.6 above, which instead uses ABCD_check_hierarchy_test and ABCD_near_threshold_test to isolate each check type.

6.8. Heavy-instance stress test

The generated instances in `data/instances/` also include heavier cases with more flights and a longer planning horizon. These runs are useful for stress-testing the implementation under a realistic exact-solver backend.

6.9. Dense heavy instances

The densest generated cases push the MILP further than the lighter stress suite. They are useful for showing where the current solver environment stops

Table 11: Heavy-instance stress test on generated **DataCplex** cases with density $\delta = 0.5$ and $|P| = 10$. With the full CPLEX Studio executable, the legacy MILP solves all three reported instances to formulation-level optimality.

Instance	MILP (legacy split)
h=7	optimal
h=15	optimal
h=21	optimal

Table 12: Dense heavy instances with density $\delta = 1.0$ and $|P| = 10$. These cases are substantially harder: the legacy MILP reports infeasibility on the smallest dense case and stops without a certificate on the larger horizons within the reported run budget.

Instance	MILP (legacy split)
h=7	infeasible
h=15	error (no certificate)
h=21	error (no certificate)
h=30	error (no certificate)

being able to certify a solution of the legacy formulation.

This dense suite shows that even with the full Studio backend, the legacy MILP reaches the practical limit of the current environment on the denser instances: the smallest dense case is reported infeasible, whereas the larger horizons terminate without a certificate inside the current run budget. These scale limits apply to the legacy exact formulation and are independent of the correctness questions addressed in Sections 6.5 and 6.6.

6.10. Summary of findings

1. The current runs validate execution on synthetic configurations named after Tables 5, 6, 10, and 11; they do not reproduce the published data or averages.
2. The Constraint (13) endpoint-split flaw of Lemma 2 is confirmed computationally: all 42 real constraint rows built by `src/model.py` for the loophole instance are satisfied at an adversarial point where 1 200 minutes are actually flown against a 600-minute limit, while the exact-state recursion correctly rejects it.
3. Solved in isolation, each of the four check types (A, B, C, D) is enforced correctly by the legacy hierarchy model on `ABCD_near_threshold_test`: optimal, fully covered, and independently validated with zero violations.
4. Under the full mixed hierarchy, the same flaw reappears unprompted on an existing test instance (aircraft 2 in `ABCD_check_hierarchy_test` exceeds its A-check threshold by 200 minutes with no violated constraint reported), showing the issue is not confined to hand-built corner cases.
5. The boundary suite confirms that the legacy endpoint-split MILP executes on all key edge cases and is infeasible on the mixed-maintenance case under CPLEX; this is a separate, solver/formulation-scale question from the correctness results above.
6. The heavy-instance stress test shows that, with the full CPLEX Studio executable, the legacy MILP solves the density-0.5 instances to formulation-level optimality.

7. On dense cases, the legacy MILP does not return a certified solution for the reported horizons within the current run budget; this is a scale limitation of the exact solver environment, not a correctness question.
8. The CPLEX installation available in this environment is a size-restricted Community/Preview edition and cannot solve any instance with maintenance constraints active (`CPLEX Error 1016`); the new Constraint (13) and hierarchy results therefore use the open-source HiGHS solver, which is exact and unrestricted.

7. Conclusions and Future Work

This paper revisited the compact Tail Assignment Problem model of Khaled et al. (2018), making three concrete contributions.

C2–C4 feasibility correction.. We showed that the basic balance constraints can admit a simultaneous- departure corner case that is mathematically feasible but operationally impossible. Adding explicit pairwise non-overlap constraints restores physical executability without altering the variable set of the compact model. This correction also clarifies the domain on which maintenance constraints are evaluated.

Constraint (13) correction.. We proved that the original cumulative-flight-time constraint can leave the opening maintenance interval unbounded. We then showed that the natural two-row endpoint split is also insufficient: it can miss flying time between consecutive check nights. The exact replacement uses a continuous daily cumulative state with maintenance-triggered resets and explicit initial conditions. The legacy computational runs use the endpoint split and therefore do not certify the exact between-check bound.

Effect on C8–C14.. The non-overlap correction changes the assignment domain on which C8–C14 operate but does not repair cumulative-hour propagation by itself. Theorem 1 therefore relies specifically on the exact state constraints, together with the corrected basic feasibility and maintenance-event constraints.

Full $\{A, B, C, D\}$ hierarchy.. We extended the single-type framework to an explicit four-level A/B/C/D hierarchy. Effective-check indicators encode heavier-check resets, separate start and occupancy variables represent multi-day C/D checks, and initial hour/day values carry maintenance history into the horizon. The extension adds $O(n_p n_d |\mathcal{C}|)$ state and event structure plus sparse flight-exclusion rows; its solve-time multiplier has not yet been established empirically.

Reproducibility.. The project contains the code, deterministic instance generator, experiment configurations, and result tables needed to rerun the reported snapshots. The legacy split formulation and the separate event/state formulation are retained as distinct implementations so their future performance can be compared without conflating their maintenance semantics.

Threats to validity.. The reported computational comparisons are intentionally scoped to the synthetic benchmark family used in this study. The quick grids use few instances, several dense runs end in infeasible/error states, and no completed paper-versus-split-versus-exact C13 experiment is reported. Performance conclusions are therefore descriptive of these runs only. Decomposition-heavy and robust/scenario alternatives may exhibit different trade-offs under their native assumptions and tuning protocols.

Future directions

- **Integrated fleet assignment and routing:** the compact model is well-suited for integration with fleet-assignment decisions, as noted by Khaled et al. (2018). The absence of the cyclic constraint facilitates this integration.
- **Robust and recoverable variants:** extending Borndörfer et al. (2010); Froyland et al. (2013) to the compact formulation with full maintenance hierarchy is an open problem.
- **Controlled formulation benchmark:** implement separately selectable paper-C13, endpoint-split, and exact-state builders, validate every solution by replaying its maintenance counters, and compare model size, relaxation bounds, runtime, and feasibility on identical instances.
- **Heuristic and metaheuristic methods:** constructive and learning-based heuristics for the corrected, full-hierarchy model are deliberately out of scope here and are left for a companion study.

References

- Barnhart, C., Belobaba, P., Odoni, A.R., 2003. Applications of operations research in the air transport industry. *Transportation Science* 37, 368–391.
- Barnhart, C., Boland, N.L., Clarke, L.W., Johnson, E.L., Nemhauser, G.L., Shenoi, R.G., 1998. Flight string models for aircraft fleetings and routing. *Transportation Science* 32, 208–220. doi:10.1287/trsc.32.3.208.

- Borndörfer, R., Dovica, I., Nowak, I., Schickinger, T., 2010. Robust tail assignment. Technical Report. Konrad-Zuse-Zentrum für Informationstechnik Berlin.
- Clarke, L., Johnson, E., Nemhauser, G., Zhu, Z., 1997. The aircraft rotation problem. *Annals of Operations Research* 69, 33–46. doi:10.1023/A:1018945415148.
- Clarke, L.W., Hane, C.A., Johnson, E.L., Nemhauser, G.L., 1996. Maintenance and crew considerations in fleet assignment. *Transportation Science* 30, 249–260.
- Cordeau, J.F., Stojković, G., Soumis, F., Desrosiers, J., 2001. Benders decomposition for simultaneous aircraft routing and crew scheduling. *Transportation Science* 35, 375–388.
- Desaulniers, G., Desrosiers, J., Dumas, Y., Solomon, M.M., Soumis, F., 1997. Daily aircraft routing and scheduling. *Management Science* 43, 841–855.
- Froyland, G., Maher, S.J., Wu, C.L., 2013. The recoverable robust tail assignment problem. *Transportation Science* 48, 351–372.
- Gopalan, R., Talluri, K.T., 1998. Mathematical models in airline schedule planning: A survey. *Annals of Operations Research* 76, 155–185.
- Grönkvist, M., 2005. The tail assignment problem. Ph.D. thesis. Chalmers, Göteborg University.
- Hane, C.A., Barnhart, C., Johnson, E.L., Marsten, R.E., Nemhauser, G.L.,

- Sigismondi, G., 1995. The fleet assignment problem: Solving a large-scale integer program. *Mathematical Programming* 70, 211–232.
- Haouari, M., Shao, S., Sherali, H.D., 2012. A lifted compact formulation for the daily aircraft maintenance routing problem. *Transportation Science* 47, 508–525.
- Hart, W.E., Laird, C.D., Watson, J.P., Woodruff, D.L., Hackebeil, G.A., Nicholson, B.L., Sirola, J.D., 2023. Pyomo – Python-based Optimization Modeling Objects. Version 6.x, <https://www.pyomo.org/>.
- Huangfu, Q., Hall, J.A.J., 2018. Parallelizing the dual revised simplex method. *Mathematical Programming Computation* 10, 119–142. HiGHS solver, <https://highs.dev/>.
- IBM, 2023. IBM ILOG CPLEX Optimization Studio: CPLEX User’s Manual. Version 22.1.
- Khaled, O., Minoux, M., Mousseau, V., Michel, S., Ceugniet, X., 2018. A compact optimization model for the tail assignment problem. *European Journal of Operational Research* 264, 548–557. doi:10.1016/j.ejor.2017.06.045.
- Klabjan, D., 2005. Large-scale models in the airline industry, in: Desaulniers, G., Desrosiers, J., Solomon, M.M. (Eds.), *Column Generation*. Springer US, pp. 163–195.
- Lacasse-Guay, E., Desaulniers, G., Soumis, F., 2010. Aircraft routing under different business processes. *Journal of Air Transport Management* 16, 258–263.

- Lavoie, S., Minoux, M., Odier, E., 1988. A new approach for crew pairing problems by column generation with an application to air transportation. *European Journal of Operational Research* 35, 45–58.
- Levin, A., 1971. Scheduling and fleet routing models for transportation systems. *Transportation Science* 5, 232–255.
- Liang, Z., Chaovalitwongse, W.A., 2012. A network-based model for the integrated weekly aircraft maintenance routing and fleet assignment problem. *Transportation Science* 47, 493–507.
- Liang, Z., Chaovalitwongse, W.A., Huang, H.C., Johnson, E.L., 2011. On a new rotation tour network model for aircraft maintenance routing problem. *Transportation Science* 45, 109–120.
- Mercier, A., Cordeau, J.F., Soumis, F., 2005. A computational study of Benders decomposition for the integrated aircraft routing and crew scheduling problem. *Computers & Operations Research* 32, 1451–1476. doi:10.1016/j.cor.2003.11.013.
- Minoux, M., 1984. Column generation techniques in combinatorial optimization, a new application to crew pairing problems, in: *Proceedings of the AGIFORS Symposium, Strasbourg, France*.
- Sarac, A., Batta, R., Rump, C.M., 2006. A branch-and-price approach for operational aircraft maintenance routing. *European Journal of Operational Research* 175, 1850–1869.
- Sherali, H.D., Bish, E.K., Zhu, X., 2006. Airline fleet assignment concepts,

models, and algorithms. *European Journal of Operational Research* 172,
1–30.